

AI-Powered Developer Assistant

A practical, student-level project explanation and interview guide. The design below describes a proposed implementation; claim each feature as completed only after building and verifying it.

1. Project goal and user experience

The assistant accepts a Git repository and a natural-language task, finds relevant code, proposes changes, runs tests, and uses failures to revise the proposal. Example: “Add email validation to the user registration endpoint and return HTTP 400 for invalid input.” A FastAPI endpoint can accept a local repository path and task:

```
POST /tasks
{"repo_path": "/workspace/sample-backend",
 "task": "Add email validation to the registration endpoint"}
```

A useful response contains completion status, modified files, a diff, test output, and a concise summary. Work on a temporary copy so the source repository remains available for review.

2. System architecture

The request enters FastAPI. LangGraph coordinates code retrieval, OpenAI patch generation, patch application, Docker-based tests, and bounded retries. RAG supplies repository context; Docker executes tests; LangGraph routes between steps. These components have distinct responsibilities.

```
Client -> FastAPI -> LangGraph workflow
                        |-> RAG retrieval -> relevant source
                        |-> OpenAI API -> proposed patch
                        |-> temporary copy + Docker -> test results
                        |-> failure -> revised patch (up to limit)
                        -> diff, status, test output
```

3. Resume bullet 1: Repository-aware code changes

Resume wording: Built an LLM-powered coding assistant using Python, OpenAI API, LangGraph, and FastAPI to analyze Git repositories and generate context-aware code changes.

FastAPI request handling

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
class CodingRequest(BaseModel):
    repo_path: str
    task: str

@app.post("/tasks")
def execute_task(request: CodingRequest):
    return coding_agent.invoke({
        "repo_path": request.repo_path,
        "task": request.task,
        "retry_count": 0,
    })
```

FastAPI validates input and provides HTTP access. A synchronous endpoint is enough for a demo; it can block while tests run. Background jobs and polling are later improvements.

Inspect the repository

Enumerate tracked source and test files, omit virtual environments, dependencies, binaries and secrets, and inspect git status and HEAD. Requiring a clean working tree simplifies patch handling. Read the requested repository from an allowlisted workspace rather than accepting arbitrary paths in a public deployment.

```
git status --short
git rev-parse HEAD
git ls-files
```

Generate a patch

Construct a prompt with the task, retrieved code, file paths, and change constraints. Ask the model for a unified diff so modifications are inspectable. Validate paths and patch format before applying to a temporary copy; reject unexpected files and patch failures.

```
TASK: Add email validation; invalid emails return HTTP 400.
CONTEXT: app/routes/users.py, app/validators.py, tests/test_users.py
RULES: Preserve existing behavior; change only relevant files.
OUTPUT: Unified diff against the provided base revision.
```

An entire updated file is easier for an initial prototype, but a patch reduces accidental changes to unrelated lines. Do not present either output as automatically safe or correct.

4. Resume bullet 2: RAG code retrieval

Resume wording: Implemented RAG-based code retrieval using semantic search to identify relevant files, functions, and dependencies for coding tasks.

Build an index

Read supported source files and split them into meaningful chunks. For Python, ast can identify top-level functions, classes and methods. Keep imports and class context available. Record path, symbol name, start/end lines, and chunk type for each segment. Large functions may need a second split.

```
{"file_path": "app/services/user_service.py",
 "symbol": "get_user_by_email",
 "start_line": 42, "end_line": 47,
 "chunk_type": "function"}
```

Generate an embedding for each chunk through the OpenAI embeddings API and store vectors in FAISS with a parallel metadata mapping. A query embedding retrieves the closest chunks; a small student repo can use a basic flat index. The meaning of similarity scores depends on the index metric.

```
from openai import OpenAI
client = OpenAI()
vec = client.embeddings.create(
    model="text-embedding-3-small", input=code_chunk
).data[0].embedding
```

```
# After adding vectors to the index:
distances, ids = index.search(query_vector, k=5)
```

Expand dependencies and tests

If retrieval finds register(), inspect its imports and referenced local symbols. Follow a direct call to create_user(), then the model or validation function it uses. Include matching tests, subject to a context budget. AST-based import and call extraction gives a best-effort relation; Python aliases, dynamic dispatch and runtime imports limit certainty.

For “prevent duplicate accounts,” semantic retrieval may find `get_user_by_email()` despite wording differences. Exact identifier search remains useful; combining both approaches is often stronger. Track file hashes or a commit ID and rebuild affected embeddings when files change, so retrieved code is current.

5. LangGraph workflow and state

LangGraph expresses the sequence as nodes with shared state and conditional transitions. The application implements each actual operation. A normal Python loop could implement the same small workflow; the graph makes the state and branches explicit.

```
from typing import TypedDict
class AgentState(TypedDict):
    repo_path: str
    task: str
    retrieved_code: str
    generated_patch: str
    test_output: str
    tests_passed: bool
    retry_count: int

from langgraph.graph import StateGraph, END
workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_code)
workflow.add_node("generate", generate_patch)
workflow.add_node("test", apply_and_test)
workflow.set_entry_point("retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", "test")
workflow.add_conditional_edges("test", decide_next_step,
    {"retry": "generate", "finish": END})
coding_agent = workflow.compile()
```

The retrieval node writes `retrieved_code`. Generation reads `task`, `context` and any previous error. Testing writes exit status and output. Route to generation only for a failed attempt below the retry limit; return a failure status if the limit is reached. Count attempts consistently.

6. Resume bullet 3: Docker tests and correction

Resume wording: Automated test execution and iterative error correction in isolated Docker environments, enabling the agent to validate and refine generated patches.

Run tests on a temporary working copy

Copy the repository at a known base revision, apply the validated patch, and run the project tests inside a container. Capture the exit code, stdout, stderr and timeout. A nonzero exit code means tests failed or execution failed; inspect the output to tell which. Build and dependency installation also execute code, so keep that process controlled.

```
docker run --rm --network none --memory 512m --cpus 1 \
  --pids-limit 128 test-runner:latest pytest -q
```

A student demo can use a prebuilt Python test image with dependencies for a known sample repository. For stronger isolation, use a non-root user, drop capabilities, bound filesystem access and execution time, and avoid host secrets, privileged mode or Docker socket mounts. Docker reduces risk but is not an absolute security boundary.

Correct a failed patch

Suppose the first patch calls `validate_email()`, while the repository actually defines `is_valid_email()`. Pytest reports `NameError`. Send the task, retrieved code, previous patch, and failure output back to the model. Ask for a complete revised patch. Reset the temporary copy to the same base revision before each attempt, then apply and test again. Stop after a bounded number of attempts, for example three corrections.

```
Attempt 1: patch uses validate_email() -> NameError
Attempt 2: patch uses is_valid_email() -> tests pass
Result: return diff, changed paths, pytest output, status
```

A passing test suite provides evidence only for behavior covered by those tests. If retries fail, return the unresolved failure and latest patch without claiming success.

7. Complete example

Task: “Add an endpoint that returns a user by email and returns HTTP 404 when missing.” FastAPI accepts the task. Retrieval finds the user route, lookup service, User model and tests. The LLM proposes a diff. The assistant applies it to a temporary copy, tests it in Docker, and either returns a passing diff or feeds the error into a revision. The final API response includes status, modified files, patch and test output.

8. Interview trade-offs

Why retrieval? A large repository may exceed the context window; targeted chunks reduce token usage and provide relevant code.

What if retrieval is wrong? Combine semantic and exact search, expand dependencies, and inspect failures to retrieve additional context.

Why function chunks? They preserve code boundaries and source locations; include class and module context when needed.

How avoid overwriting edits? Check Git status and work on a copy anchored to the original revision.

How validate correctness? Run existing and task-specific tests, inspect the diff, and acknowledge untested cases.

Why Docker? It supplies a controlled runtime and resource limits for test execution; stronger isolation is needed for untrusted production workloads.

Why cap retries? Bound runtime and API cost and make unresolved failures explicit.

9. A conversational interview explanation

“I built a coding assistant that takes a natural-language task for a Python Git repository and proposes code changes. The main challenge was providing relevant context without sending the entire repo to the model. I indexed functions and files with embeddings, retrieved the relevant chunks, and expanded direct dependencies and tests. FastAPI receives the request, while LangGraph coordinates retrieval, patch generation, test execution and retries. Each patch is applied to a temporary copy and tested in Docker. If tests fail, the error output informs a revised patch, with a retry limit. The result includes the diff and test status.”

10. One-week implementation scope

- FastAPI POST endpoint for a local Python Git repository and task

- Python source indexing with AST chunks and file/symbol metadata
- OpenAI embeddings plus a FAISS index and retrieval
- Best-effort direct dependency and relevant-test expansion
- Patch generation and validation against an isolated working copy
- LangGraph retrieval, generation, test, and conditional retry nodes
- Docker-based pytest execution with resource and time limits
- Diff, modified file list, test output, and explicit success/failure status

Demonstrate two or three small tasks, including one where a failed first patch is corrected. Keep the README honest about supported languages, test dependencies, isolation limits, and which features are implemented.